

Reviewer Pack — Spectral Concentration of Moment Matrices in Max-CUT SOS Hie...

Entry 5167cdc13e84 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 21:59:43 UTC

Spectral Concentration of Moment Matrices in Max-CUT SOS Hierarchy

Entry ID: 5167cdc13e84 Verdict: INCONCLUSIVE Recorded: 2026-05-10 21:59:43 UTC

1. Conjecture

Field A (mathematical branch): Random Matrix Theory **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For a random Max-CUT instance with n variables, the spectral norm of the degree- d moment matrix M_d is at least $\Omega(\sqrt{n})$ if $d < c \log n$ for some constant c . Thus, any SOS proof system achieving a 0.879-approximation for Max-CUT must have $d \geq \Omega(\log n)$.

Rationale (proposer’s reasoning):

The spectral norm of the moment matrix reflects the ability to capture the structure of the Max-CUT problem. If the norm is too small, the SOS hierarchy cannot approximate the problem well, implying a lower bound on the required degree.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c57d7a13661baabc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from typing import List, Dict

def run_trial(seed: int) -> Dict[str, any]:
    random.seed(seed)

    def generate_max_cut_instance(n: int) -> List[List[int]]:
        return [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    def matrix_multiply(A: List[List[float]], B: List[List[float]]) -> List[List[float]]:
        n = len(A)
        C = [[0.0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def spectral_norm(M: List[List[float]]) -> float:
        n = len(M)
        v = [1.0 / math.sqrt(n)] * n
        for _ in range(100): # Power iteration method
            v_next = matrix_multiply(M, v)
            norm_v_next = sum(x**2 for x in v_next) ** 0.5
            v = [x / norm_v_next for x in v_next]
        return abs(v[0])

    def moment_matrix(instance: List[List[int]], d: int) -> List[List[float]]:
        n = len(instance)
        M = [[0.0] * (n + 1) for _ in range(n + 1)]
        M[0][0] = 1.0
        for i in range(n):
            for j in range(i, n):
                M[i+1][j+1] = instance[i][j]
                M[j+1][i+1] = instance[i][j]
        for k in range(2, d + 1):
            new_M = [[0.0] * (n + 1) for _ in range(n + 1)]
            for i in range(n):
                for j in range(i, n):
```

```

        for l in range(j, n):
            new_M[i+1][j+1] += instance[i][l] * M[j+1][k-1+l]
            new_M[j+1][i+1] += instance[i][l] * M[j+1][k-1+l]
    M = new_M
    return M

n = random.randint(5, 40)
instance = generate_max_cut_instance(n)
d = 5
M_d = moment_matrix(instance, d)
spectral_norm_M_d = spectral_norm(M_d)

if spectral_norm_M_d < 0.9 * math.sqrt(n):
    return {
        "metric_name": "spectral_norm",
        "metric_value": spectral_norm_M_d,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "spectral norm too small"
    }
else:
    return {
        "metric_name": "spectral_norm",
        "metric_value": spectral_norm_M_d,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='spectral norm too small' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12e5d29b.py", line 89, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12e5d29b.py", line 63, in run_trial
    M_d = moment_matrix(instance, d)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12e5d29b.py", line 55, in moment_matrix
    new_M[i+1][j+1] += instance[i][l] * M[j+1][k-1+l]
                        ~~~~~^~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before collecting data; cannot assess conjecture validity |
next: Debug the moment_matrix implementation to fix index out-of-range error

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	93930
2	propose	ollama_remote	qwen3:8b	0	0	111700
3	propose	ollama_remote	qwen3:8b	0	0	84140
4	preregistration	ollama_remote	qwen3:8b	0	0	25379
5	novelty	ollama_remote	qwen3:8b	0	0	20677
6	novelty	ollama_remote	qwen3:8b	0	0	15028
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13081
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11004
9	judge	ollama_remote	qwen3:8b	0	0	15423

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 390362 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/5167cdc13e84.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5167cdc13e84.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5167cdc13e84.tar.gz` (if generated)